

Prima esercitazione — Rappresentazione dell'informazione

1. Bit e codifiche

Il calcolatore rappresenta le informazioni attraverso sequenze di bit. La stessa sequenza può indicare un numero, un carattere o un colore: per sapere che cosa significa, bisogna conoscere la **codifica**

utilizzata. [\[pplxfilegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws.com\]](https://pplxfilegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws.com)

Bit e byte

- Un bit può assumere due valori: 0 oppure 1.
- Questi valori possono rappresentare informazioni come falso/vero o spento/acceso.
- Un byte è composto da 8 bit.
- Con n bit si possono costruire 2^n configurazioni diverse. [\[pplxfilegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws.com\]](https://pplxfilegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws.com)

Per esempio, con 3 bit:

000 001 010 011 100 101 110 111

Sono 8 configurazioni. Se le interpretiamo come numeri naturali, rappresentano i valori da 0 a 7. Quindi bisogna distinguere:

$\text{Numero di configurazioni} = 2^n$

$\text{Massimo naturale rappresentabile} = 2^n - 1$

Non c'è contraddizione: anche lo zero occupa una configurazione. Con un byte si rappresentano 256 naturali, da 0 a

255. [\[ppl-ai-file-upload.s3.amazonaws.com\]](https://ppl-ai-file-upload.s3.amazonaws.com) [\[pplxfilegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws.com\]](https://pplxfilegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws.com)

Quanti bit servono?

Per rappresentare K simboli diversi con codici tutti della stessa lunghezza, bisogna trovare il minimo intero n tale che:

$$2^n \geq K$$

Perciò:

$$n = \left\lceil \log_2 K \right\rceil$$

Esempio: alfabeto di 195 simboli.

$$2^7=128<195, \text{ quindi } 2^8=256 \geq 195$$

Servono 8 bit. Alcune configurazioni rimarranno inutilizzate, ma tutti i simboli avranno un codice. [\[ppl-ai-file-upload.s3.amazonaws\]](#)

Attenzione alla differenza tra queste due richieste:

Richiesta	Bit necessari	Motivo
Rappresentare 32 simboli diversi	5	$2^5=32$ configurazioni
Rappresentare tutti i naturali da 0 a 32	6	Sono 33 valori
Rappresentare il naturale 32 in binario	6	$32_{10}=100000_2$

Per un naturale $N \geq 0$, la lunghezza della sua rappresentazione binaria è:

$$\left\lceil \log_2 N \right\rceil + 1$$

Questa formula non va confusa con quella per il numero di simboli. [\[ppl-file-gateway-projectfiles-prod-use1.s3.us-east-1.amazonaws\]](#)

Parte intera e arrotondamento superiore

- $\lfloor x \rfloor$, detto floor: il più grande intero non maggiore di x .
- $\lceil x \rceil$, detto ceiling: il più piccolo intero non minore di x . [\[ppl-ai-file-upload.s3.amazonaws\]](#)

x	$\lfloor x \rfloor$	$\lceil x \rceil$
5,3	5	6
5	5	5

Quindi ceiling e floor differiscono di 1 soltanto quando il numero non è intero. Questo evita l'errore di assegnare 5 bit a 16 simboli: ne bastano 4. [\[ppl-ai-file-upload.s3.amazonaws\]](#)

Testo e immagini

Nel materiale del corso, la codifica ASCII associa ai caratteri valori numerici. I tre riferimenti consigliati dall'esercitatore sono:

Carattere	Codice decimale
'0'	48
'A'	65
'a'	97

Le cifre, le lettere maiuscole e le lettere minuscole sono consecutive nei rispettivi gruppi: per esempio '1' vale 49 e 'b' vale 98. Attenzione: il carattere '0' non coincide con il numero 0.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

Le immagini trattate nella lezione sono composte da pixel. Se ogni pixel dispone di n bit per codificare il colore, può distinguere fino a 2^n colori: 1 bit permette 2 colori, 10 bit ne permettono 1024. Il voxel è l'equivalente volumetrico del pixel nella rappresentazione tridimensionale.[\[ppl-ai-file-upload.s3.amazonaws\]](#)[\[pplxfilegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws\]](#)

Esercizio sulle date

La lezione mostra che il numero di bit dipende anche da come organizziamo l'informazione:

Codifica scelta	Bit
Mese + settimana nel mese + giorno della settimana	$4+3+3=10$
Numero progressivo del giorno nell'anno	9
Mese + giorno del mese	$4+5=9$

La prima soluzione assume una convenzione per identificare le settimane nel mese; non è la codifica globale più compatta della data. Per distinguere tutti i 365 giorni servono almeno 9 bit, perché $2^8=256$ non basta, mentre $2^9=512$ sì. Anche con 366 giorni il risultato non cambia.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

2. Basi e conversioni

Notazione posizionale

In base b , ogni cifra viene moltiplicata per una potenza della base, determinata dalla sua posizione. Partendo da destra, le posizioni della parte intera sono 0, 1, 2, \ldots [\[ppl-ai-file-upload.s3.amazonaws\]](#)

$$(d_k \dots d_1 d_0)_b = \sum_{i=0}^k d_i b^i$$

Per esempio:

$$145_{10} = 1 \cdot 10^2 + 4 \cdot 10^1 + 5 \cdot 10^0$$

La stessa regola vale in binario:

$$\begin{aligned} 101101_2 \\ &= 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 \\ &= 45_{10} \end{aligned}$$

Il punto fondamentale è che una cifra ha un valore diverso a seconda della posizione. [\[ppl-ai-file-upload.s3.amazonaws\]](#)

Da decimale a un'altra base

Per convertire la parte intera si usa la divisione ripetuta per la base di destinazione:

1. Dividi il numero per la base.
2. Annota quoziente e resto.
3. Ripeti sul quoziente.
4. Fermati quando il quoziente diventa 0.
5. Leggi i resti dal basso verso l'alto. [\[ppl-ai-file-upload.s3.amazonaws\]](#)

Esempio svolto: convertire 792_{10} in base 5.

Divisione	Quoziente	Resto
$792 : 5$	158	2
$158 : 5$	31	3
$31 : 5$	6	1
$6 : 5$	1	1
$1 : 5$	0	1

Leggendo i resti al contrario:

$$\boxed{792_{10} = 11132_5}$$

Il docente suggerisce di leggere le cifre singolarmente: «uno, uno, uno, tre, due, in base cinque», evitando di interpretarle come un numero decimale. [\[ppl-ai-file-upload.s3.amazonaws\]](#)

Come controllare il risultato

Si torna in decimale usando i pesi posizionali:

$$\begin{aligned} &11132_5 \\ &= 1 \cdot 5^4 + 1 \cdot 5^3 + 1 \cdot 5^2 + 3 \cdot 5 + 2 \\ &= 625 + 125 + 25 + 15 + 2 = 792 \end{aligned}$$

Se ritrovi il numero iniziale, la conversione è corretta. [\[ppl-ai-file-upload.s3.amazonaws\]](#)

Scorciatoia con le potenze di 2

Per numeri piccoli puoi scomporre direttamente il valore:

$$35 = 32 + 2 + 1 = 2^5 + 2^1 + 2^0$$

Quindi accendi i bit nelle posizioni 5, 1 e 0:

Pesi: 32 16 8 4 2 1
Bit: 1 0 0 0 1 1
 $\boxed{35}_{10} = 100011_2$

È un'alternativa alla divisione ripetuta, non una codifica diversa. [\[ppl-ai-file-upload.s3.amazonaws\]](#)

3. Numeri interi con segno

La sequenza di bit, da sola, non basta: bisogna conoscere sia il numero di bit sia la rappresentazione utilizzata. Le tre codifiche studiate sono modulo e segno, complemento a 1 e complemento a 2. [\[pplxfilegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws\]](#)

Modulo e segno

Su n bit:

- Il bit più a sinistra rappresenta il segno: 0 per positivo, 1 per negativo.
- Gli altri $n-1$ bit rappresentano il valore assoluto. [\[pplxfilegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws\]](#)

Esempio su 5 bit:

+15 → 01111
-15 → 11111

Esistono due codifiche dello zero:

+0 → 00000
-0 → 10000

Perciò le configurazioni sono 2^n , ma i valori numerici distinti sono $2^n - 1$.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

Complemento a 1

Per rappresentare il negativo di un numero positivo, si invertono tutti i bit della sua rappresentazione, già scritta sulla lunghezza corretta:

+15 → 01111
↓ inverti tutti i bit
-15 → 10000

Anche qui lo zero ha due codifiche:

+0 → 00000
-0 → 11111

Per leggere un negativo, puoi invertire nuovamente i bit, ricavare il valore positivo e applicare il segno meno.[\[ppl-ai-file-upload.s3.amazonaws\]](#)[\[pplxfilegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws\]](#)

Complemento a 2

Per ottenere l'opposto di un numero:

1. Inverti tutti i bit.
2. Somma 1.
3. Conserva soltanto gli n bit previsti.[\[pplxfilegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws\]](#)

Esempio:

+15	01111
Inversione	10000
Somma di 1	+ 00001

-15	10001

Metodo alternativo equivalente:

1. Scorri da destra verso sinistra.
2. Lascia invariati i bit fino al primo 1 compreso.
3. Inverti tutti i bit rimanenti a sinistra.[\[pplxfilegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws\]](#)

Il complemento a 2 ha una sola codifica dello zero e un intervallo non simmetrico: dispone di un valore negativo in più rispetto ai positivi.[\[pplxfilegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws\]](#)

Intervalli rappresentabili

Codifica su n bit	Minimo	Massimo	Valori distinti
Naturali senza segno	0	$2^n - 1$	2^n
Modulo e segno	$-(2^{n-1} - 1)$	$2^{n-1} - 1$	2^{n-1}
Complemento a 1	$-(2^{n-1} - 1)$	$2^{n-1} - 1$	2^{n-1}
Complemento a 2	-2^{n-1}	$2^{n-1} - 1$	2^n

Con 8 bit: senza segno da 0 a 255; modulo e segno o complemento a 1 da -127 a +127; complemento a 2 da -128 a

+127. [\[ppl-ai-file-upload.s3.amazonaws.com/pplxfilegateway-projectfiles-prod-use1.s3.us-east-1.amazonaws.com\]](https://ppl-ai-file-upload.s3.amazonaws.com/pplxfilegateway-projectfiles-prod-use1.s3.us-east-1.amazonaws.com)

Il peso del bit iniziale

Chiamiamo s il bit più significativo e L il valore senza segno rappresentato dagli altri $n-1$ bit.

Codifica	Valore
Modulo e segno	$(-1)^s L$
Complemento a 1	$-s(2^{n-1} - 1) + L$
Complemento a 2	$-s2^{n-1} + L$

Quindi nel complemento a 2 il bit iniziale non è semplicemente un segno separato: ha peso negativo -2^{n-1} . Su 6 bit i pesi sono:

-32 16 8 4 2 1

Nel complemento a 1, invece:

-31 16 8 4 2 1

Questa differenza spiega perché una stessa configurazione negativa vale un'unità in meno in complemento a 2 rispetto al complemento a 1. [\[ppl-ai-file-upload.s3.amazonaws.com/pplxfilegateway-projectfiles-prod-use1.s3.us-east-1.amazonaws.com\]](https://ppl-ai-file-upload.s3.amazonaws.com/pplxfilegateway-projectfiles-prod-use1.s3.us-east-1.amazonaws.com)

Esercizio: codificare -35

Prima:

$35_{10} = 100011_2$

Per le tre codifiche usiamo 7 bit:

Passaggio o codifica	Risultato
+35 su 7 bit	0100011
-35, modulo e segno	1100011
-35, complemento a 1	1011100
-35, complemento a 2	1011101

In complemento a 1 si inverte 0100011; in complemento a 2 si aggiunge 1 al risultato.[\[ppl-ai-file-upload.s3.amazonaws.com\]](https://ppl-ai-file-upload.s3.amazonaws.com)

Esercizio: interpretare 1000101

La trascrizione perde alcune cifre, ma i risultati indicati corrispondono alla sequenza di 7 bit 1000101.

Interpretazione	Calcolo	Valore
Modulo e segno	Segno negativo, modulo 000101	-5
Complemento a 1	-63+4+1	-58
Complemento a 2	-64+4+1	-59

È l'esempio più chiaro del fatto che stessi bit non significa stesso numero.[\[ppl-ai-file-upload.s3.amazonaws.com\]](https://ppl-ai-file-upload.s3.amazonaws.com)

4. Somme e overflow

Operazioni elementari

Somma	Bit risultante	Riporto
0+0	0	0
0+1	1	0
1+0	1	0
1+1	0	1

Con un riporto precedente, $1+1+1=11_2$: si scrive 1 e si riporta 1. Per il prodotto di due bit, soltanto $1 \cdot 1$ dà 1; gli altri casi danno

0. [\[ppl-ai-file-upload.s3.amazonaws.com/pplxfilegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws.com\]](#)

Prima della somma

Gli operandi devono essere rappresentati sullo stesso numero di bit. Se il testo assegna una lunghezza, devi rispettarla e verificare che ciascun operando sia rappresentabile. [\[ppl-ai-file-upload.s3.amazonaws.com\]](#)

Per estendere un numero già in complemento a 2, si ripete il bit iniziale:

+5: 0101 $\rightarrow$ 000101

-5: 1011 $\rightarrow$ 111011

Lo si verifica dai pesi del complemento a 2: entrambi i passaggi conservano il valore. Non si aggiungono quindi sempre zeri a sinistra; per un negativo si aggiungono uno. [\[ppl-ai-file-upload.s3.amazonaws.com\]](#)

Esempio: $27 + (-12)$

Su 6 bit:

+27 $\rightarrow$ 011011

+12 $\rightarrow$ 001100

-12 $\rightarrow$ 110100

Somma:

```

011011
+ 110100
-----
1 001111

```

Si scarta il riporto esterno e si conserva:

001111₂ = 15₁₀

Quindi $27 - 12 = 15$. Il riporto esterno non indica, da solo, un overflow con segno. [\[ppl-ai-file-upload.s3.amazonaws\]](#)

Esempio: $11 + (-17)$

```
+11 → 001011
+17 → 010001
-17 → 101111
  001011
+ 101111
-----
 111010
```

Lettura con i pesi:

$111010_{\text{C2}} = -32 + 16 + 8 + 2 = -6$

Il risultato è corretto: $11 - 17 = -6$. [\[ppl-ai-file-upload.s3.amazonaws\]](#)

Che cos'è l'overflow?

L'overflow si verifica quando il risultato matematico dell'operazione è fuori dall'intervallo rappresentabile con i bit disponibili. [\[ppl-ai-file-upload.s3.amazonaws\]](#)

Nella somma in complemento a 2:

- Due positivi che producono un risultato con bit iniziale 1 indicano overflow.
- Due negativi che producono un risultato con bit iniziale 0 indicano overflow.
- Due operandi di segno opposto non producono overflow nella somma. [\[pplxfilegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws\]](#)

Esempio su 6 bit, il cui intervallo è $[-32, 31]$:

$25 + 18 = 43$

43 non è rappresentabile.

```
011001
+ 010010
-----
 101011
```

La configurazione ottenuta, interpretata in complemento a 2, vale:

$-32 + 8 + 2 + 1 = -21$

Non è il risultato corretto: è l'effetto del traboccamento. [\[ppl-ai-file-upload.s3.amazonaws\]](#) [\[pplxfilegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws\]](#)

Un controllo equivalente confronta il riporto in ingresso al bit più significativo con quello in uscita:

$\boxed{\text{Overflow se i due riporti sono diversi}}$

Non basta guardare se esiste un riporto

finale. [\[pplxfilelegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws\]](https://pplxfilelegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws.com)

5. Frazioni e IEEE 754

Pesi dopo la virgola

La notazione posizionale continua anche dopo la virgola, usando esponenti negativi:

$$2^{-1} = \frac{1}{2}$$

$$2^{-2} = \frac{1}{4}$$

$$2^{-3} = \frac{1}{8}$$

Per esempio:

$$10.01_2$$

$$= 1 \cdot 2^1 + 1 \cdot 2^{-2}$$

$$= 2,25_{10}$$

Il primo bit frazionario vale metà, il secondo un quarto, il terzo un ottavo e così via. [\[pplxfilelegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws\]](https://pplxfilelegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws.com)

Virgola fissa

In virgola fissa, la posizione della virgola è stabilita: alcuni bit rappresentano la parte intera e altri quella frazionaria. Per convertire un numero:

- Parte intera: divisioni ripetute.
- Parte frazionaria: moltiplicazioni ripetute. [\[ppl-ai-file-upload.s3.amazonaws\]](https://ppl-ai-file-upload.s3.amazonaws.com/[pplxfilelegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws]) [\[pplxfilelegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws\]](https://pplxfilelegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws.com)

Procedimento per la frazione:

1. Moltiplica la frazione per 2.
2. Annota la parte intera del prodotto.
3. Conserva soltanto la parte frazionaria.
4. Ripeti.
5. Leggi le parti intere nell'ordine in cui le hai ottenute, dall'alto verso il basso. [\[ppl-ai-file-upload.s3.amazonaws\]](https://ppl-ai-file-upload.s3.amazonaws.com)

Esempio: $0,145_{10}$, limitandosi a 4 bit frazionari.

Passaggio	Prodotto	Bit	Frazione successiva
$0,145 \cdot 2$	0,290	0	0,290
$0,290 \cdot 2$	0,580	0	0,580
$0,580 \cdot 2$	1,160	1	0,160
$0,160 \cdot 2$	0,320	0	0,320

Quindi:

$0,145_{10} \approx 0,0010_2$

Il valore effettivo di $0,0010_2$ è $2^{-3} = 0,125$: si tratta di un'approssimazione, non di un'uguaglianza esatta. [\[ppl-ai-file-upload.s3.amazonaws\]](#)

Il procedimento termina esattamente quando la frazione residua diventa zero. Altrimenti si continua oppure ci si ferma alla precisione richiesta. Aggiungere zeri dopo una conversione troncata non recupera i bit mancanti. [\[ppl-ai-file-upload.s3.amazonaws\]](#)

Virgola mobile

Un numero viene espresso separando significando ed esponente:

$$x = m \cdot b^e$$

Nel caso binario normalizzato usato per IEEE 754:

$$x = (-1)^S \cdot 1.F_2 \cdot 2^e$$

La virgola viene spostata fino a lasciare un solo 1 prima di essa; l'esponente compensa lo spostamento. [\[pplxfilegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws\]](#)

È utile distinguere:

- **S**: bit di segno.
- **e**: esponente effettivo.
- **E**: esponente memorizzato.
- **F**: bit della frazione memorizzati, chiamati spesso "campo mantissa" nel corso.
- **1.F₂**: significando completo, con l'1 iniziale implicito. [\[pplxfilegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws\]](#)

Struttura IEEE 754

Formato	Bit totali	Segno	Esponente	Frazione
Precisione singola	32	1	8	23
Precisione doppia	64	1	11	52

Per i numeri normalizzati in precisione singola:

$$E = e + 127$$

$$x = (-1)^S \cdot (1.F)_2 \cdot 2^{E-127}$$

L'esponente memorizzato non è in complemento a 2: usa la rappresentazione in eccesso 127. I valori 0 e 255 del campo esponente sono riservati a casi speciali, quindi la formula precedente non si applica indistintamente a tutte le configurazioni. [[pplxfileitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws](#)]

Esempio completo: -118,5

Nel PDF l'esempio è -118,5, anche se in un punto della trascrizione compare -18,5. [[ppl-ai-file-upload.s3.amazonaws](#)][[pplxfileitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws](#)]

1. Segno: il numero è negativo, quindi $S=1$.
2. Conversione del valore assoluto:

$$118 = 64 + 32 + 16 + 4 + 2 = 1110110_2$$

$$0,5 = 0,1_2$$

Quindi:

$$118,5 = 1110110,1_2$$

3. Normalizzazione:

$$1110110,1_2 = 1,1101101_2 \cdot 2^6$$

4. Esponente memorizzato:

$$E = 6 + 127 = 133 = 10000101_2$$

5. Campo frazione: si elimina l'1 implicito e si completa fino a 23 bit.

$$11011010000000000000000$$

La codifica finale è:

1 | 10000101 | 110110100000000000000000

Le barre servono soltanto a distinguere i campi. [\[pplxfileitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws\]](#)

Come decodificare

Per leggere una configurazione IEEE 754 normalizzata in precisione singola:

1. Leggi il primo bit: determina il segno.
2. Leggi gli 8 bit successivi: ricava E .
3. Calcola $e = E - 127$.
4. Leggi i 23 bit frazionari e ricostruisci $1.F_2$.
5. Moltiplica per 2^e e applica il segno. [\[ppl-ai-file-upload.s3.amazonaws\]](#) [\[pplxfileitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws\]](#)

Nell'esempio:

$$x = -1.1101101_2 \cdot 2^6 = -1110110.1_2 = -118,5$$

La conversione è esatta perché la frazione $0,5$ termina in binario. [\[pplxfileitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws\]](#)

Esempio approssimato: -75,42

Il docente converte $0,42$ usando soltanto 4 bit:

Passaggi o	Prodott o	Bit
$0,42 \cdot 2$	0,84	0
$0,84 \cdot 2$	1,68	1
$0,68 \cdot 2$	1,36	1
$0,36 \cdot 2$	0,72	0

Quindi:

$$0,42_{10} \approx 0.0110_2$$

Ma:

$$0.0110_2 = \frac{14}{2^6} = 0,375$$

La conversione preliminare rappresenta dunque $75,375$, non $75,42$.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

Normalizzando:

$$\begin{aligned} 75,375 &= 1001011.0110_2 \\ &= 1.0010110110_2 \cdot 2^6 \end{aligned}$$

La codifica costruita a lezione, completando con zeri, è:

1 | 10000101 | 001011011000000000000000

Questa rappresenta esattamente $-75,375$. È una codifica IEEE 754 valida del valore approssimato ottenuto, ma non va presentata come la conversione a piena precisione di $-75,42$: l'errore è stato introdotto prima, fermando la frazione dopo 4 bit.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

Errori da evitare

- Confondere 2^n configurazioni con il massimo naturale $2^n - 1$.[\[pplxfilegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws\]](#)
- Leggere i resti delle divisioni dall'alto verso il basso: vanno letti al contrario.[\[ppl-ai-file-upload.s3.amazonaws\]](#)
- Leggere al contrario i bit prodotti dalle moltiplicazioni: vanno mantenuti nell'ordine ottenuto.[\[ppl-ai-file-upload.s3.amazonaws\]](#)
- Complementare un numero prima di aver fissato la lunghezza corretta.[\[ppl-ai-file-upload.s3.amazonaws\]](#)
- Sommare operandi con lunghezze diverse.[\[pplxfilegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws\]](#)
- Confondere riporto esterno e overflow con segno.[\[pplxfilegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws\]](#)
- Memorizzare l'1 iniziale nel campo frazione IEEE 754: per i numeri normalizzati è implicito.[\[pplxfilegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws\]](#)
- Usare E direttamente come esponente effettivo senza sottrarre 127.[\[pplxfilegitgateway-projectfiles-prod-use1.s3.us-east-1.amazonaws\]](#)
- Scrivere $=$ quando hai troncato una frazione: in quel passaggio serve $\approx$.[\[ppl-ai-file-upload.s3.amazonaws\]](#)